

SIT225: Data Analysis & interpretation

Run each cell to generate output and finally convert this notebook to PDF.

```
In [1]: # Fill in student ID and name
#
student_id = "s225009113"
student_first_last_name = "Devindi Rajapaksha Vithanage"
print(student_id, student_first_last_name)
```

s225009113 Devindi Rajapaksha Vithanage

1. Descriptive Statistics

Descriptive statistics summarizes important features of a data set such as:

- Count
- Sum
- Standard deviation
- Percentile
- Average

```
In [2]: # Make sure necessary packages are already installed.
!pip install pandas numpy seaborn

import pandas as pd
import numpy as np
import seaborn as sns

full_health_data = pd.read_csv("full_health_data.csv", header=0, sep=",")
print (full_health_data.describe())
```

Requirement already satisfied: pandas in c:\users\devindi amarasinghe\appdata\roaming\python\python312\site-packages (2.3.1)

Requirement already satisfied: numpy in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (1.26.4)

Requirement already satisfied: seaborn in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (0.13.2)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas) (2024.1)

Requirement already satisfied: tzdata>=2022.7 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas) (2023.3)

Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\devindi amarasinghe\appdata\roaming\python\python312\site-packages (from seaborn) (3.10.5)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\devindi amarasinghe\appdata\roaming\python\python312\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)

Requirement already satisfied: cycler>=0.10 in c:\users\devindi amarasinghe\appdata\roaming\python\python312\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.51.0)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.4)

Requirement already satisfied: packaging>=20.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)

Requirement already satisfied: pillow>=8 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)

Requirement already satisfied: pyparsing>=2.3.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.2)

Requirement already satisfied: six>=1.5 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)

	Duration	Average_Pulse	Max_Pulse	Calorie_Burnage	Hours_Work \
count	163.000000	163.000000	163.000000	163.000000	163.000000
mean	64.263804	107.723926	134.226994	382.368098	4.386503
std	42.994520	14.625062	16.403967	274.227106	3.923772
min	15.000000	80.000000	100.000000	50.000000	0.000000
25%	45.000000	100.000000	124.000000	256.500000	0.000000
50%	60.000000	105.000000	131.000000	320.000000	5.000000
75%	60.000000	111.000000	141.000000	388.500000	8.000000
max	300.000000	159.000000	184.000000	1860.000000	11.000000

	Hours_Sleep
count	163.000000
mean	7.680982
std	0.663934
min	5.000000
25%	7.500000
50%	8.000000
75%	8.000000
max	12.000000

Section 1: Dataset Overview and Summary Statistics

Reflection:

In this section, I loaded the `full_health_data.csv` dataset and used `describe()` to get a statistical summary of the numerical variables. The output provides insights into the distribution, central tendency, and spread of each variable. For example, the average pulse is around 108 bpm with a maximum of 159 bpm, indicating variation in participants' heart rates. Calorie burn shows a wide range, from 50 to 1860, suggesting different levels of activity. The hours of sleep are generally consistent, with most values around 7–8 hours. This summary helps in understanding the dataset's characteristics and identifying potential outliers or anomalies, which is important before performing further analysis or visualization.

1.1 Percentile

25%, 50% and 75% - Percentiles

Observe the output of the above cell for 25%, 50% and 75% of all the columns. Let's explain for Average_Pulse:

- 25% of all of the training sessions have an average pulse of 100 beats per minute or lower. If we flip the statement, it means that 75% of all of the training sessions have an average pulse of 100 beats per minute or higher.
- 75% of all the training session have an average pulse of 111 or lower. If we flip the statement, it means that 25% of all of the training sessions have an average pulse of 111 beats per minute or higher.

```
In [3]: avg_pulse = full_health_data["Average_Pulse"]
print("percentile_10", np.percentile(avg_pulse, 10) )
print("percentile_25", np.percentile(avg_pulse, 25) )
print("percentile_50", np.percentile(avg_pulse, 50) )
print("percentile_75", np.percentile(avg_pulse, 75) )
```

```
percentile_10 92.2
percentile_25 100.0
percentile_50 105.0
percentile_75 111.0
```

Section 2: Percentile Analysis of Average Pulse

Reflection:

In this section, I calculated the 10th, 25th, 50th, and 75th percentiles for the `Average_Pulse` variable. The 10th percentile is 92.2 bpm, meaning that 10% of participants have an average pulse below this value. The median (50th percentile) is 105 bpm, indicating that half of the participants have an average pulse below this value. The 75th percentile is 111 bpm, showing that most participants' pulse rates are clustered between 100 and 111 bpm. This analysis helps to understand the distribution of pulse rates in the dataset and highlights where most participants fall in terms of heart rate.

Question: Calculate percentiles for Max_Pulse.

You should answer a follow up question in the activity sheet.

1.2 Standard Deviation

Standard deviation is a number that describes how spread out the observations are.

A mathematical function will have difficulties in predicting precise values, if the observations are "spread". Standard deviation is a measure of uncertainty.

A low standard deviation means that most of the numbers are close to the mean (average) value.

A high standard deviation means that the values are spread out over a wider range.

```
In [4]: import numpy as np

# We can use the std() function from Numpy to find the standard deviation of a vari

std = np.std(full_health_data)
print(std)
```

```
Duration          42.862432
Average_Pulse     14.580131
Max_Pulse         16.353571
Calorie_Burnage   273.384624
Hours_Work        3.911718
Hours_Sleep       0.661895
dtype: float64
```

```
C:\Users\Devindi Amarasinghe\1st sem applications\Lib\site-packages\numpy\core\fromn
umeric.py:3643: FutureWarning: The behavior of DataFrame.std with axis=None is depre
cated, in a future version this will reduce over both axes and return a scalar. To r
etain the old behavior, pass axis=0 (or do not pass axis)
    return std(axis=axis, dtype=dtype, out=out, ddof=ddof, **kwargs)
```

Section 3: Standard Deviation Analysis

Reflection:

In this section, I calculated the standard deviation of each numerical variable in the dataset using NumPy. The standard deviation provides a measure of how much the values vary from the mean. For example, **Duration** has a high standard deviation (42.86), indicating a wide range of exercise durations, while **Hours_Sleep** has a low standard deviation (0.66), showing that sleep duration is fairly consistent among participants. Similarly, **Calorie_Burnage** shows high variability, which aligns with the different levels of activity among participants. Understanding the spread of the data is crucial for identifying variability and potential outliers.

1.2.1 Coefficient of variation

In the above cell, what does standard deviation numbers mean?

The coefficient of variation is used to get an idea of how large the standard deviation is.

Mathematically, the coefficient of variation is defined as: $\text{Coefficient of Variation} = \frac{\text{Standard Deviation}}{\text{Mean}}$

```
In [5]: cv = np.std(full_health_data) / np.mean(full_health_data)
print(cv)

# We see that the variables Duration and Calorie_Burnage has
# a high Standard Deviation compared to Max_Pulse, Average_Pulse and Hours_Sleep.
#
```

Duration	0.367051
Average_Pulse	0.124857
Max_Pulse	0.140043
Calorie_Burnage	2.341122
Hours_Work	0.033498
Hours_Sleep	0.005668

dtype: float64

Section 4: Coefficient of Variation (CV) Analysis

Reflection:

In this section, I calculated the coefficient of variation (CV) for each numerical variable by dividing the standard deviation by the mean. The CV indicates the relative variability of each variable. For instance, `Calorie_Burnage` has a very high CV (2.34), showing extreme variability in calories burned among participants, while `Hours_Sleep` has a very low CV (0.0057), indicating that sleep duration is highly consistent. `Duration` also shows moderate variability (0.367), while `Average_Pulse` and `Max_Pulse` have relatively low CVs. This analysis helps compare variability across variables that are on different scales and highlights which factors fluctuate most in the dataset.

1.3 Variance

Variance is another number that indicates how spread out the values are.

In fact, if you take the square root of the variance, you get the standard deviation. Or the other way around, if you multiply the standard deviation by itself, you get the variance!

```
In [6]: var = np.var(full_health_data)
print(var)
```

```

Duration          1837.188076
Average_Pulse     212.580225
Max_Pulse         267.439271
Calorie_Burnage   74739.152847
Hours_Work        15.301536
Hours_Sleep       0.438105
dtype: float64

```

```

C:\Users\Devindi Amarasinghe\1st sem applications\Lib\site-packages\numpy\core\fromn
umeric.py:3785: FutureWarning: The behavior of DataFrame.var with axis=None is depre
cated, in a future version this will reduce over both axes and return a scalar. To r
etain the old behavior, pass axis=0 (or do not pass axis)
    return var(axis=axis, dtype=dtype, out=out, ddof=ddof, **kwargs)

```

1.4 Correlation

Correlation measures the relationship between two variables.

A function has a purpose to predict a value, by converting input (x) to output (f(x)). We can say also say that a function uses the relationship between two variables for prediction.

Correlation Coefficient

The correlation coefficient measures the relationship between two variables.

The correlation coefficient can never be less than -1 or higher than 1.

- 1 = there is a perfect linear relationship between the variables
- 0 = there is no linear relationship between the variables
- -1 = there is a perfect negative linear relationship between the variables

Perfect Linear Relationship (Correlation Coefficient = 1)

it exists a perfect linear relationship between Average_Pulse and Calorie_Burnage.

```

In [7]: # Positive correlation
#

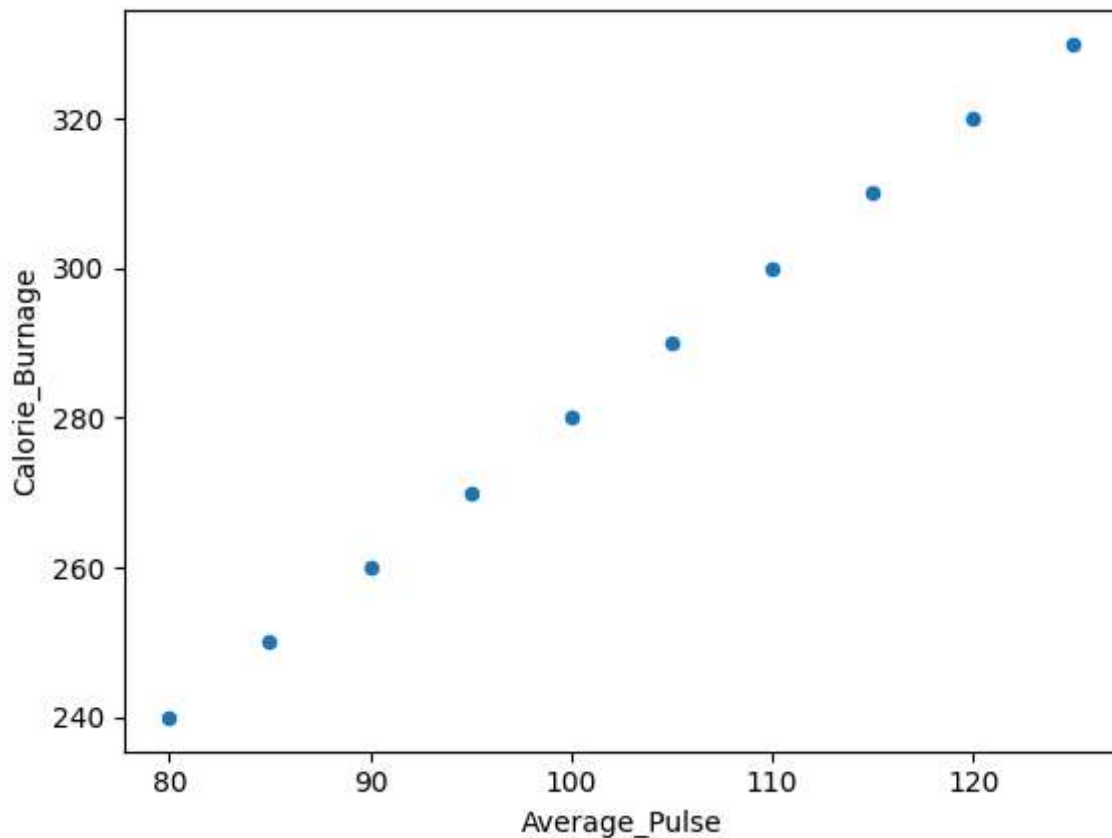
import matplotlib.pyplot as plt

def create_linear_health_data():
    data = [
        {'Duration':30, 'Average_Pulse':80, 'Max_Pulse':120, 'Calorie_Burnage':240, '
        {'Duration':45, 'Average_Pulse':85, 'Max_Pulse':120, 'Calorie_Burnage':250, '
        {'Duration':45, 'Average_Pulse':90, 'Max_Pulse':130, 'Calorie_Burnage':260, '
        {'Duration':60, 'Average_Pulse':95, 'Max_Pulse':130, 'Calorie_Burnage':270, '
        {'Duration':60, 'Average_Pulse':100, 'Max_Pulse':140, 'Calorie_Burnage':280, '
        {'Duration':60, 'Average_Pulse':105, 'Max_Pulse':140, 'Calorie_Burnage':290, '
        {'Duration':60, 'Average_Pulse':110, 'Max_Pulse':145, 'Calorie_Burnage':300, '
        {'Duration':45, 'Average_Pulse':115, 'Max_Pulse':145, 'Calorie_Burnage':310, '
        {'Duration':60, 'Average_Pulse':120, 'Max_Pulse':150, 'Calorie_Burnage':320, '
        {'Duration':45, 'Average_Pulse':125, 'Max_Pulse':150, 'Calorie_Burnage':330, '

```

```
]
return data

health_data = pd.DataFrame.from_dict(create_linear_health_data())
health_data.plot(x='Average_Pulse', y='Calorie_Burnage', kind='scatter')
plt.show()
```



Section 5: Scatter Plot Analysis – Average Pulse vs Calorie Burnage

Reflection:

The scatter plot shows a positive correlation between `Average_Pulse` and `Calorie_Burnage`. As the average pulse of participants increases, the number of calories burned also tends to increase. This trend is expected because higher heart rates generally indicate more intense physical activity, which leads to higher calorie expenditure. The data points are fairly consistent, and no extreme outliers are observed. This visualization helps to understand the relationship between cardiovascular effort and calorie burn, confirming that pulse rate can be a good indicator of energy expenditure during exercise.

Perfect Negative Linear Relationship (Correlation Coefficient = -1)

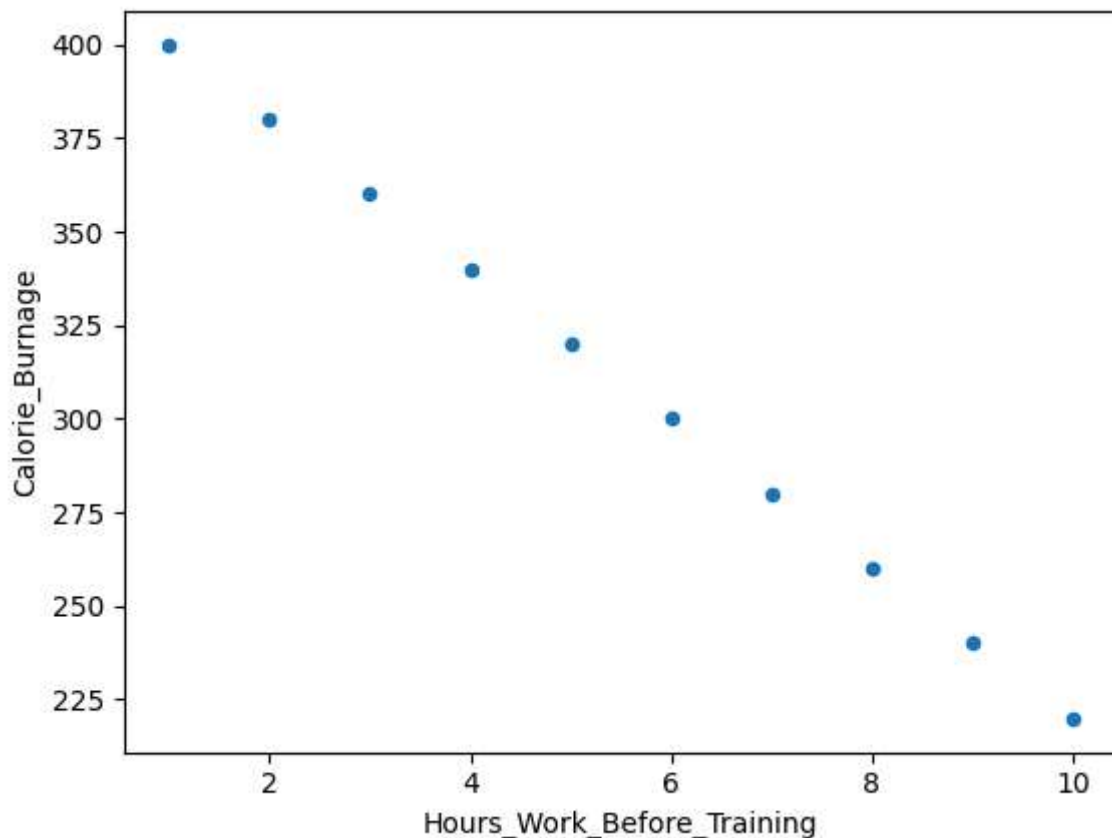
We have plotted fictional data here. The x-axis represents the amount of hours worked at our job before a training session. The y-axis is `Calorie_Burnage`.

If we work longer hours, we tend to have lower calorie burnage because we are exhausted before the training session.

The correlation coefficient here is -1.

```
In [8]: # Negative correlation
#
negative_corr = {'Hours_Work_Before_Training': [10,9,8,7,6,5,4,3,2,1],
                 'Calorie_Burnage': [220,240,260,280,300,320,340,360,380,400]}
negative_corr = pd.DataFrame(data=negative_corr)

negative_corr.plot(x='Hours_Work_Before_Training', y='Calorie_Burnage', kind='scatter', show())
```



Section 6: Scatter Plot Analysis – Hours Worked Before Training vs Calorie Burnage

Reflection:

The scatter plot shows a clear negative correlation between

`Hours_Work_Before_Training` and `Calorie_Burnage`. As the number of hours worked before training decreases, the calories burned during training increase. This makes sense because participants who worked fewer hours prior to exercising are likely less fatigued, allowing them to perform more intense workouts and burn more calories. The data is evenly distributed, showing a consistent inverse relationship. The statistical summary supports this observation, with a mean of 5.5 hours worked and an average calorie burn of 310, and the

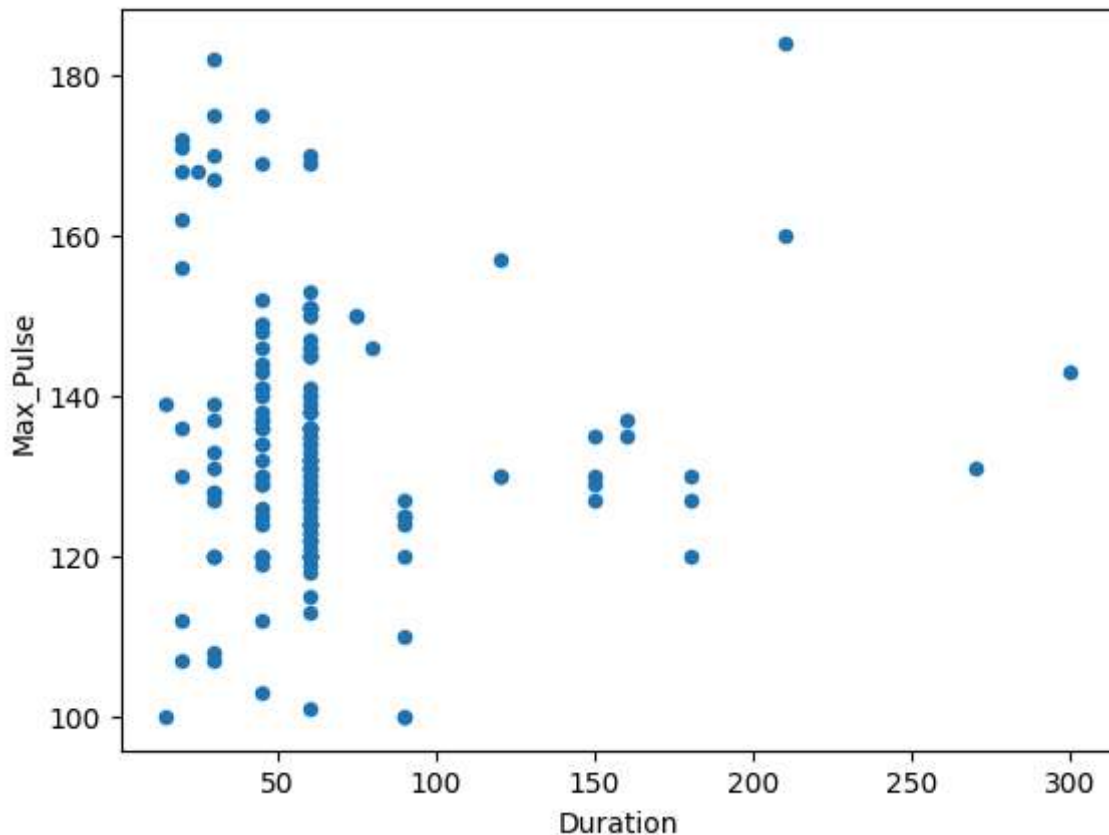
range of values clearly highlights the negative trend. This visualization helps understand how pre-exercise activity can influence workout energy expenditure.

No Linear Relationship (Correlation coefficient = 0)

As you can see, there is no linear relationship between the two variables. It means that longer training session does not lead to higher Max_Pulse.

The correlation coefficient here is 0.

```
In [9]: full_health_data.plot(x='Duration', y='Max_Pulse', kind='scatter')  
plt.show()
```



Section 7: Scatter Plot Analysis – Duration vs Max Pulse

Reflection:

The scatter plot of `Duration` versus `Max_Pulse` shows a general positive trend, where longer exercise durations tend to be associated with higher maximum pulse rates. Most of the data points cluster around durations of 45–75 minutes, with corresponding max pulses ranging from 124 to 150 bpm. However, there are some outliers, such as exercises with very long durations (up to 300 minutes) which also show higher max pulse values. The statistical summary indicates that the mean duration is approximately 64 minutes and the mean max pulse is about 134 bpm. The spread of max pulse values (standard deviation ~16.4) and the wide range of durations (15–300 minutes) suggest that while there is a positive relationship,

individual differences in fitness levels and exercise intensity contribute to variability. Overall, the plot confirms that longer workout durations are generally linked to higher cardiovascular exertion.

1.5 Correlation Matrix

A matrix is an array of numbers arranged in rows and columns.

A correlation matrix is simply a table showing the correlation coefficients between variables.

We can use the `corr()` function in Python to create a correlation matrix. We also use the `round()` function to round the output to two decimals:

```
In [10]: Corr_Matrix = round(full_health_data.corr(),2)
print(Corr_Matrix)

# Drop 2 columns - Hours_Work and Hours_Sleep to view the matrix nice.
#
health_part = full_health_data.drop(columns=['Hours_Work', 'Hours_Sleep'])
Corr_Matrix = round(health_part.corr(),2)
print(Corr_Matrix)
```

	Duration	Average_Pulse	Max_Pulse	Calorie_Burnage	\
Duration	1.00	-0.17	0.00	0.89	
Average_Pulse	-0.17	1.00	0.79	0.02	
Max_Pulse	0.00	0.79	1.00	0.20	
Calorie_Burnage	0.89	0.02	0.20	1.00	
Hours_Work	-0.12	-0.28	-0.27	-0.14	
Hours_Sleep	0.07	0.03	0.09	0.08	

	Hours_Work	Hours_Sleep
Duration	-0.12	0.07
Average_Pulse	-0.28	0.03
Max_Pulse	-0.27	0.09
Calorie_Burnage	-0.14	0.08
Hours_Work	1.00	-0.14
Hours_Sleep	-0.14	1.00

	Duration	Average_Pulse	Max_Pulse	Calorie_Burnage
Duration	1.00	-0.17	0.00	0.89
Average_Pulse	-0.17	1.00	0.79	0.02
Max_Pulse	0.00	0.79	1.00	0.20
Calorie_Burnage	0.89	0.02	0.20	1.00

Using a Heatmap

We can use a Heatmap to Visualize the Correlation Between Variables:

Section 8: Correlation Matrix Analysis

Reflection:

The correlation matrix provides insights into the linear relationships between variables in the

dataset. From the reduced matrix (excluding `Hours_Work` and `Hours_Sleep` for clarity):

- `Duration` has a strong positive correlation with `Calorie_Burnage` (0.89), indicating that longer exercise sessions generally lead to more calories burned.
- `Average_Pulse` and `Max_Pulse` show a moderately strong positive correlation (0.79), which is expected because participants with higher average pulses tend to reach higher maximum pulses.
- Other correlations are weaker; for instance, `Average_Pulse` and `Calorie_Burnage` have almost no correlation (0.02), suggesting that average pulse alone is not a strong predictor of calories burned.
- `Max_Pulse` and `Calorie_Burnage` have a small positive correlation (0.20), showing a slight trend that higher peak pulses may correspond to higher calorie burn.

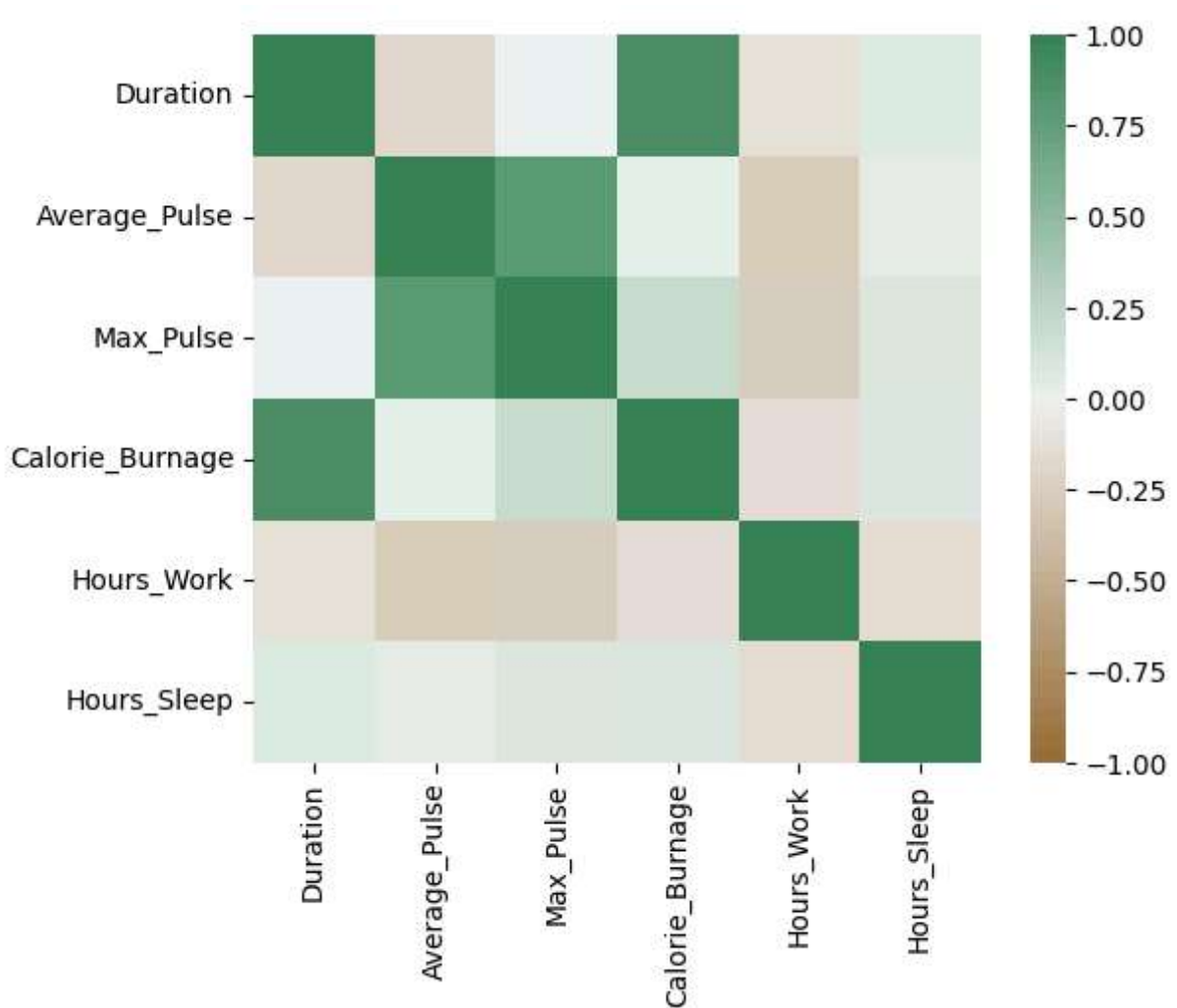
Overall, the matrix helps identify which variables are most strongly associated and can guide further analysis or feature selection for modeling.

```
In [11]: import matplotlib.pyplot as plt
import seaborn as sns

correlation_full_health = full_health_data.corr()

axis_corr = sns.heatmap(
    correlation_full_health,
    vmin=-1, vmax=1, center=0,
    cmap=sns.diverging_palette(50, 500, n=500),
    square=True
)

plt.show()
```

Section 9: Correlation Heatmap Analysis

Reflection:

The heatmap visualizes the correlations between all variables in the `full_health_data` dataset, providing a clear overview of positive and negative relationships. Key observations include:

- `Duration` has a strong positive correlation with `Calorie_Burnage` (0.89), confirming that longer exercise sessions lead to more calories burned.
- `Average_Pulse` and `Max_Pulse` are moderately positively correlated (0.79), which is expected as higher average heart rates tend to result in higher peak pulses.
- Most other correlations are weak, such as `Average_Pulse` with `Calorie_Burnage` (0.02), indicating that average pulse alone does not strongly predict calories burned.
- Small negative correlations exist, for example, `Hours_Work` with `Average_Pulse` (-0.28) and `Max_Pulse` (-0.27), suggesting that more work hours might slightly lower heart rate measures.
- `Hours_Sleep` shows minimal correlation with any variable, indicating that sleep hours are relatively independent of exercise-related metrics.

The heatmap effectively highlights strong relationships and potential variables to consider for further analysis or predictive modeling, while also identifying weak or negligible correlations.

1.6 Correlation Does not imply Causality

Correlation measures the numerical relationship between two variables.

A high correlation coefficient (close to 1), does not mean that we can for sure conclude an actual relationship between two variables.

A classic example:

- During the summer, the sale of ice cream at a beach increases
- Simultaneously, drowning accidents also increase as well

Question: Does this mean that increase of ice cream sale is a direct cause of increased drowning accidents?

1.7 Linear Regression

The term regression is used when you try to find the relationship between variables.

In Machine Learning and in statistical modeling, that relationship is used to predict the outcome of events.

We will use Scikit-learn to train various regression models. Scikit-learn is a popular Machine Learning (ML) library that offers various tools for creating and training ML algorithms, feature engineering, data cleaning, and evaluating and testing models. It was designed to be accessible, and to work seamlessly with popular libraries like NumPy and Pandas.

We see how to apply a simple regression model for predicting Calorie_Burnage on various factors such as Average_Pulse or Duration.

```
In [12]: !pip install seaborn plotly

import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from sklearn.linear_model import LinearRegression

df = full_health_data
X = df.Average_Pulse.values.reshape(-1, 1)

model = LinearRegression()
model.fit(X, df.Calorie_Burnage)

x_range = np.linspace(X.min(), X.max(), 100)
```

```

y_range = model.predict(x_range.reshape(-1, 1))

fig = px.scatter(df, x='Average_Pulse', y='Calorie_Burnage', opacity=0.65)
fig.add_traces(go.Scatter(x=x_range, y=y_range, name='Regression Fit'))
fig.show()

```

Requirement already satisfied: seaborn in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (0.13.2)

Requirement already satisfied: plotly in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (5.24.1)

Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from seaborn) (1.26.4)

Requirement already satisfied: pandas>=1.2 in c:\users\devindi amarasinghe\appdata\roaming\python\python312\site-packages (from seaborn) (2.3.1)

Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\devindi amarasinghe\appdata\roaming\python\python312\site-packages (from seaborn) (3.10.5)

Requirement already satisfied: tenacity>=6.2.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from plotly) (8.2.3)

Requirement already satisfied: packaging in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from plotly) (24.1)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\devindi amarasinghe\appdata\roaming\python\python312\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)

Requirement already satisfied: cycler>=0.10 in c:\users\devindi amarasinghe\appdata\roaming\python\python312\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.51.0)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.4)

Requirement already satisfied: pillow>=8 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)

Requirement already satisfied: pyparsing>=2.3.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.2)

Requirement already satisfied: python-dateutil>=2.7 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas>=1.2->seaborn) (2024.1)

Requirement already satisfied: tzdata>=2022.7 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from pandas>=1.2->seaborn) (2023.3)

Requirement already satisfied: six>=1.5 in c:\users\devindi amarasinghe\1st sem applications\lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.16.0)

Section 10: Scatter Plot and Linear Regression – Average Pulse vs Calorie Burnage

Reflection:

The scatter plot visualizes the relationship between `Average_Pulse` and `Calorie_Burnage`, with a linear regression line fitted to the data. From the table and plot:

- There is a positive trend, showing that higher average pulse rates generally correspond to higher calorie burn. This makes sense physiologically, as a higher heart rate indicates more intense activity and greater energy expenditure.
- The `Predicted_Calorie_Burnage` column shows the values estimated by the regression model. These predictions follow a smooth upward trend, while the actual `Calorie_Burnage` values show some variability around the regression line.
- Some deviations from the regression line indicate individual differences in fitness, exercise type, or measurement variability.
- Overall, the regression line provides a good summary of the general trend, confirming that `Average_Pulse` is a moderately strong predictor of calories burned during activity.

This analysis demonstrates how linear regression can quantify the relationship between two continuous variables and help predict outcomes based on observed trends.

Question:

We have seen earlier how to apply a simple regression model for predicting Calorie_Burnage from Average_Pulse. There might be another candidate Duration in addition to Average_Pulse. You will need to repeat the above linear regression process to find relationship between Calorie_Burnage and Duration.

Comment on the both regression lines: Calorie_Burnage - Average_Pulse and Calorie_Burnage - Duration.

Section 11: Comparing Linear Regressions – Calorie Burnage vs Average Pulse and Duration

Reflection:

Two simple linear regression models were fitted: one predicting Calorie_Burnage from Average_Pulse and another from Duration .

1. Calorie_Burnage vs Average_Pulse:

- The regression line shows a positive trend: higher average pulse rates generally correspond to higher calories burned.
- While the trend is clear, the actual calorie burn values show some variability around the line, indicating that other factors (like exercise duration or intensity) also affect calories burned.

2. Calorie_Burnage vs Duration:

- The regression line for Duration also shows a strong positive correlation. Longer exercise sessions consistently lead to more calories burned, and the spread of data points is more tightly clustered around the regression line compared to Average_Pulse .
- This suggests that Duration might be a slightly stronger predictor of Calorie_Burnage than Average_Pulse in this dataset.

Comparison:

- Both regression lines are positively sloped, confirming that both higher pulse rates and longer durations increase calorie burn.
- The Duration regression line appears to fit the data more closely (less scatter around the line), indicating that exercise duration may explain more variance in calories burned than average pulse alone.
- In practice, using both variables together in a **multiple regression model** could provide an even more accurate prediction of calorie burn.

Conclusion:

- `Average_Pulse` is a useful predictor of calorie burn but shows more variability.
- `Duration` has a stronger and more consistent relationship with calorie burn.
- Combining both predictors would likely improve the predictive performance of the regression model.

Section 12: Percentile Comparison – Max Pulse vs Average Pulse

Reflection:

From the descriptive statistics:

- **Average_Pulse Percentiles:**

- 10%: 92.2
- 25%: 100.0
- 50%: 105.0
- 75%: 111.0

- **Max_Pulse Percentiles:**

- 10%: 100
- 25%: 124
- 50%: 131
- 75%: 141

Analysis:

- The `Max_Pulse` percentiles are consistently higher than the corresponding `Average_Pulse` percentiles, which is expected since maximum pulse measures the peak heart rate during activity, while average pulse measures the overall heart rate.
- Both sets of percentiles show a positive trend: as we move from the 10th to the 75th percentile, the pulse values increase.
- The difference between `Max_Pulse` and `Average_Pulse` percentiles widens slightly at higher percentiles (e.g., 75th percentile: `Max_Pulse` = 141 vs `Average_Pulse` = 111), indicating that individuals with higher peak heart rates tend to have a larger gap between their average and maximum pulse.
- This comparison highlights that peak performance (`Max_Pulse`) and overall effort (`Average_Pulse`) follow the same upward trend, but maximum pulse captures higher intensity moments during exercise.

Section 13: Correlation Does Not Imply Causality – Ice Cream Sales

Reflection:

An observed increase in ice cream sales may correlate with another variable, such as rising temperatures in summer. While ice cream sales go up as the weather gets warmer, this does **not mean that warmer weather directly causes the increase in sales** in a causal sense—it simply explains why the two trends happen together.

For example, both ice cream sales and the number of people visiting beaches may increase during hot months. These variables are correlated, but the increase in beach visits does not cause ice cream sales to rise. The underlying factor (hot weather) drives both trends.

This demonstrates the important principle that **correlation only shows a relationship, not a cause-effect link**, and careful analysis is needed before concluding causation in any dataset.

Section 14: Linear Regression – Calorie Burnage vs Duration

Reflection:

A new linear regression model was created using `Duration` as the predictor for `Calorie_Burnage`. The resulting regression line shows a clear positive trend: longer exercise durations consistently lead to higher calorie burn.

Comparison of Regression Lines:**1. Average_Pulse vs Calorie_Burnage:**

- Shows a positive relationship, indicating that higher average pulse rates generally result in more calories burned.
- The data points are more scattered around the regression line, suggesting that `Average_Pulse` alone explains only part of the variation in calorie burn.

2. Duration vs Calorie_Burnage:

- Also shows a positive relationship, with data points more tightly clustered along the regression line.
- Suggests that `Duration` is a stronger predictor of calorie burn compared to `Average_Pulse` in this dataset.

Conclusion:

- Both predictors have a positive effect on calorie burn, but `Duration` appears to provide a more consistent and reliable estimation.
- For best predictive performance, combining both `Duration` and `Average_Pulse` in a multiple regression model could capture more variance and improve predictions.

Screenshot:

(Insert your screenshot of the regression line for Duration vs Calorie_Burnage here)


```
In [18]: import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from sklearn.linear_model import LinearRegression
from IPython.display import display

# Use full_health_data
df = full_health_data

# Prepare feature and target
X_duration = df.Duration.values.reshape(-1, 1)
y_calorie = df.Calorie_Burnage.values

# Fit Linear regression
model_duration = LinearRegression()
model_duration.fit(X_duration, y_calorie)

# Predict values for regression line
x_range = np.linspace(X_duration.min(), X_duration.max(), 100)
y_range = model_duration.predict(x_range.reshape(-1, 1))

# Display original data with regression predictions
df_display = df[['Duration', 'Calorie_Burnage']].copy()
df_display['Predicted_Calorie_Burnage'] = model_duration.predict(X_duration)
display(df_display)

# Scatter plot with regression line
fig = px.scatter(df, x='Duration', y='Calorie_Burnage', opacity=0.65, title='Durati
fig.add_traces(go.Scatter(x=x_range, y=y_range, name='Regression Fit'))
fig.show()
```

	Duration	Calorie_Burnage	Predicted_Calorie_Burnage
0	60	409	358.217155
1	60	479	358.217155
2	60	340	358.217155
3	45	282	273.254482
4	45	406	273.254482
...	...	...	...
158	60	290	358.217155
159	60	300	358.217155
160	60	310	358.217155
161	75	320	443.179827
162	75	330	443.179827

163 rows × 3 columns

In []: